Lec 15 3D Vision.

- Today: - Far from Deeplearning: What is 3D Vision?
  - How AI has changed 3D Vision?
  - How to integrated 整合 in different ways?

- Represent Geometry 几何表示.

  - Explicit 显式: 定向、显式地表示物体的一部分.
    - point cloud 点云
    - Polygon mesh 多边形网格.
    - Subdivision. NURBS ☆. 细分网格.
  - Implict 隐含:
    - Level sets. 水平集
    - Algebraic surface 代数曲面
    - Distance functions 距离函数.

每种选择都有其优劣势.

Considerations:

1. Store 易储.
2. Creation of shapes. 可扩展. 创建新形状 (输入语言. 图像?).
3. Operation 编辑. 操作.
4. Rendering 渲染. 转化为2d.
5. Animation 动画.

- Point Cloud.

最简单的表示.

- 仅有点. 无连接.
- (x.y.z)的集合.
- 有时带有法线方向的有向点. (用于渲染时处理光线).

- 大多数扫描. 感知的原始输出
- 会非常noisy. 需处理
- 需将多张切片联合生成.
- 灵活, 可以表示几乎所有形状.
- 采样需要"均匀." 没有简化和细分
- 不允许平滑渲染.
- 没有拓扑信息.

- 多边形网格. Polygonal Meshes.



- 包含点和对象, 以及点之间的连接.
- 表示"点"和"表面".
- 最广泛使用的3d对象表示. (图像引擎. 电子游戏...)

- 可细分: 使用更多网格细化. 也可简化.
- 正则化.

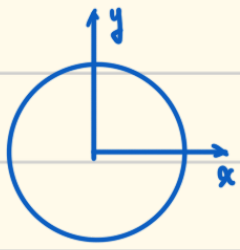
其他的表示方式（显式）.

- Parametric Representation. 参数表示.

$f: X \to Y$. $X \subseteq \mathbb{R}^m$. $Y \subseteq \mathbb{R}^n$.

$S(u,v) = (x(u,v), y(u,v), z(u,v))$

e.g.:



$p: \mathbb{R} \to \mathbb{R}^2$ 参数t. 函数p(t).

$t \longmapsto p(t) = (x(t), y(t))$.

$p(t) = r(\cos(t), \sin(t))$. $t \in [0, 2\pi)$.

其他表示曲线.面的参数表示:

- 贝塞尔曲线. 分段贝塞尔.
- 贝塞尔三角量积.



$S: \mathbb{R}^2 \to \mathbb{R}^3$

$S(u,v) = r(\cos(u)\cos(v), \sin(u)\cos(v), \sin(v))$.

$(u,v) \in [0, 2\pi) \times [-\pi/2, \pi/2]$.

- 支持细分.

"Explicit" 显式表示总结:

- 所有点都直接给出. (点 → 表面表示 → 所有点).
  - → 采样非常简单. (例如球体，仅需随机采样(u,v).)
- Hard: Inside/Outside Test 难以确认某点是否在内/外部.

- "Implicit" Representations 隐式表示

核心: "分类"点.

- 点之间通常满足特定关系.

e.g. 球体: $x^2+y^2+z^2=1$ "约束"条件

⇓

更广泛地. 约束条件是 $f(x,y,z)=0$. → 其约束本身表示了一个图形.

Hard: 更难以采样. 例如: $f(x,y,z) = (2-\sqrt{x^2+y^2})^2 + z^2 - 1$

要采样(x,y,z)就要解方程.

Easy: 测试在内/外. $f(x,y,z) > 0 / < 0$.

- 隐含表示怎样处理复杂实体：

逻辑操作：结合，交集，差集 ⟶ 组合成为复杂图形.



- 距离函数.

利用距离函数，渐变混合表面.

插值 ← 

组成复杂图形，产生变形功能.

- Level Set Methods. 水平集方法（隐式）.

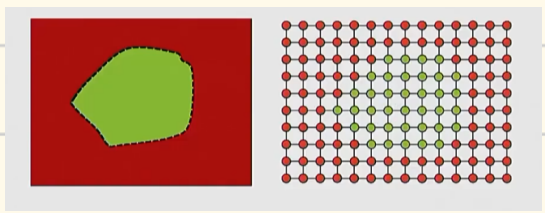
4	3	2	2	3	2
3	2	1	1	0.5	0.5
1.5	1	0.5	0	-0.5	-1
0.5	0	-1	-1	-2	-2
-1	-1	-2	-2	-3	-3

f(0)=0. 约束函数.

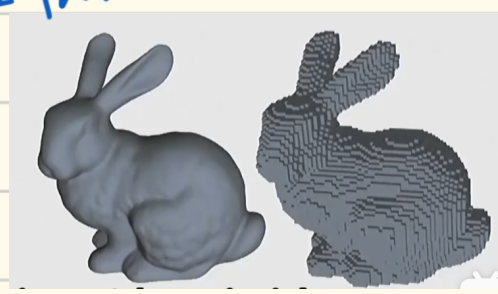
以一定尺度建立矩阵，储存其与边界的距离.

（数据储存）.

通过数据储存，隐式表示也变为显式表示.



二进制0/1矩阵，产生"体素"表示



2012年，AlexNet在2维图像上的成功促使计算机图形学的相关从业者开始考虑三维图像在卷积模型上应用的可能性。从观图形表示方法，"体素"表示是最接近像素，最适合3d Conv的表示方法..

（但实际上相比点云或多边形，Voxel非常糟糕（计算速度很慢，依靠隐含表示）。从Voxels开始仅因为其与2d Conv的代码相当匹配）.

AI + Geometry: Datasets.

- Princeton Shape Benchmark.

- 814个模型
- 182种类别.

⟹ 单个类别的数据相当小

在2014年前都有这样的问题.

- ShapeNet. 3M models.
- ShapeNetCore 51.3k models in 55 categories. (不平均，有很多椅子).

- Objaverse (800k). 更多模型，更高质量/纹理，更平均.
- Objaverse-XL (10M).

CO3D ...

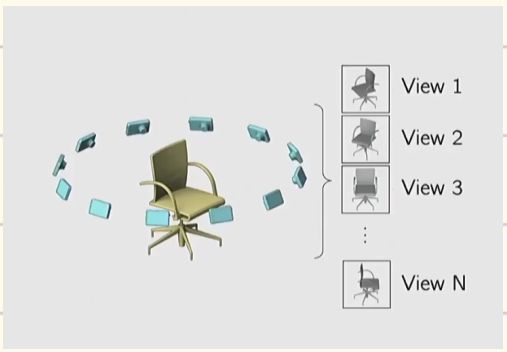
↘ Datasets for { parts, move, Scenes. (much smaller).

{ 3D对象数据集的规模始终是巨大挑战.
但如今的数据已足够训练模型.

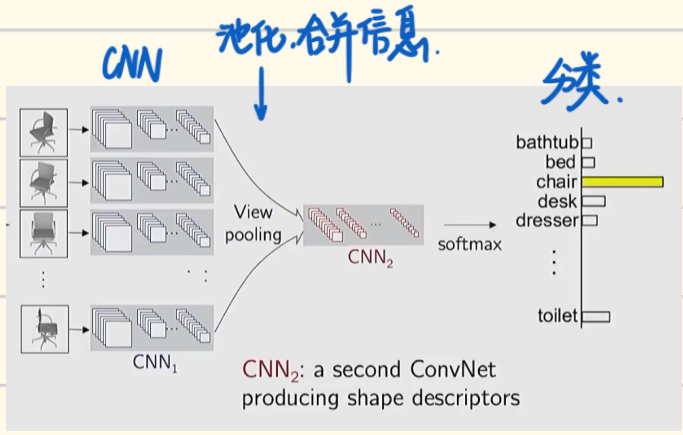
AI + Geometry: Task:

- P(S) / P(S|C): 生成模型.
  - (条件) 补形先验学习.
  ...
- P(C|S): 分辨别模型.
- 联合2D, 3D训练. (利用2D先验).
- 多维度信息联合训练.

- Multi-View CNN: 最初的分辨别方式.



不同视角渲染
⇓
2d问题.



在分类任务上表现非常好.

优势: { 性能良好.
可以利用已经非常优秀，丰富的图像处理文献
可使用2D预训练特征.

劣势: { 需要2D投影(损失信息).
Input可能会很noisy (例如点云等).

- Pixels → Voxels. 3D原生方法.

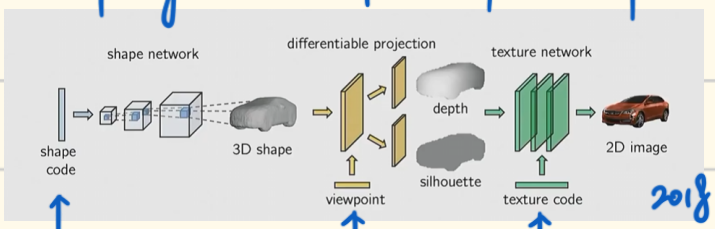
  · 3D Conv Deep Belief Networks (CDBN).

  (生成网络. 也可用于分类.)

- 3D-GANs. (8-9年前).

  能给出一个优秀的模型.

Visual Object Networks. (建模+渲染).



2018

形状 视角 纹理.

得到既3维相似. 又投影相似的生成模型.

- Octree Representations 八度树表示. (2019).

  假设基础的体素可以拥有不同的大小.

  在物体表面会以更精细的尺度描绘物体.

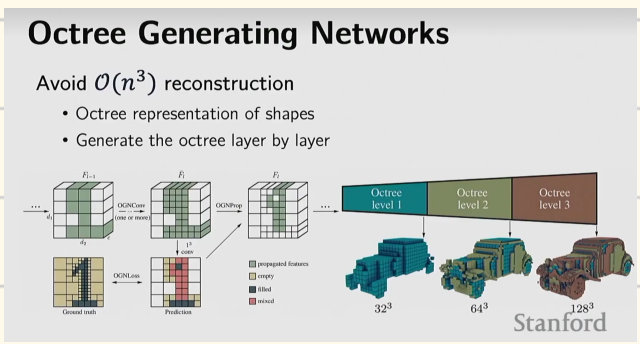
↓

- 递归地划分空间.
- 在内存上有巨大优势.

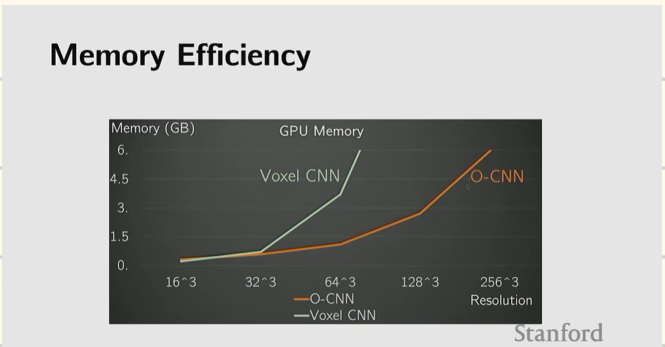
↓

(OGN).

- Octree Generating Networks. 八度树生成网络



多级生成. 相同分辨得到更精细的模型.



↓ 非Voxels 模型.

- Point Net: 在点云上学习

key: { 点云没有空间的隐含信息. 是无序的点 → 排列不变性.
       不论采样的密度. 方式如何. 应当得到相同结果 → 采样不变性.

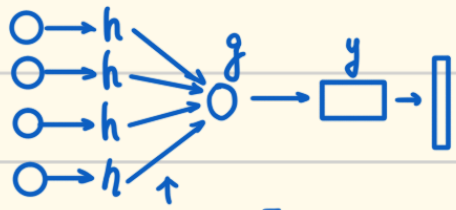
⇓

排列不变性：对称函数. 嵌入点云输入.

(例如 $f(x_1, x_2, \cdots, x_n) = \max(x_1, x_2, \cdots)$

或 $f(x_1, x_2 \cdots, x_n) = x_1 + x_2 + \cdots + x_n$)

↓ 应用至神经网络：



PointNet (vanilla). 事实证明，非常强大.

↑ 对称函数，聚合.

- 改进：
  - Points → Nodes.
  - Neighborhood → Edges.

- loss / distance for Point Cloud. 对点云生成模型，怎样计算 loss？
  - Chamfer distance：对两个点集 S_1, S_2：

    对任意 S_1 中点，找出其与 S_2 中最近的点并计算距离，求出"平方和" d_1.

    对 $\cdots S_2, \cdots S_1, \cdots, d_2$.

    $$d_{CD}(S_1, S_2) = d_1 + d_2 = \sum_{x \in S_1} \min_{y \in S_2} \|x - y\|_2 + \sum_{y \in S_2} \min_{x \in S_1} \|y - x\|_2$$

  - Earth Mover's distance：

    假设点集 S_1, S_2 有相同大小 $|S_1| = |S_2|$.

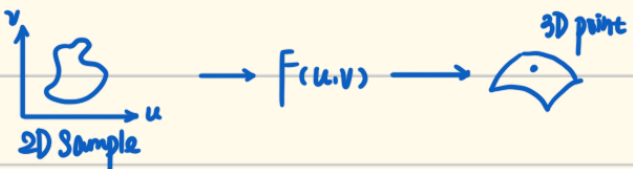
    任找一个映射 $\phi: S_1 \to S_2$ (一一对应. 双射).

    $$d_{EMD}(S_1, S_2) = \min_{\phi: S_1 \to S_2} \sum_{x \in S_1} \|x - \phi(x)\|_2.$$

    → 可微，以计算梯度.

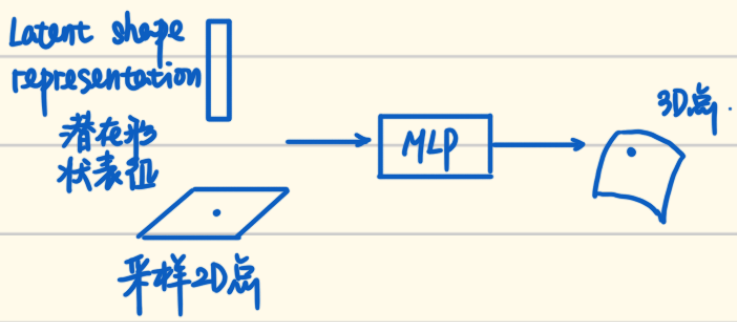
- Parametric Decoder：AtlasNet.

  idea：尝试用神经网络构建参数表示. (光滑表面.)

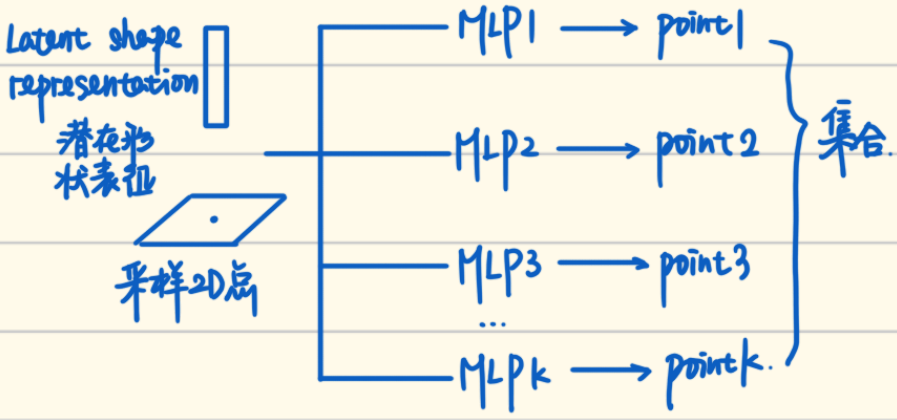


对参数表示，$F(u,v)$ 很难是一个封闭的函数形式。

正因如此，可以尝试用神经网络取代 $F(u,v)$.



但是人们认为单个 MLP 用来代替整个模型太困难，可以用多个小形的网络来表示.



- Deep Implicit Functions.

一直以来，神经网络都运行在模型的像素或类似的介质上，但是能否更深入些，将网络变成隐式函数本身？

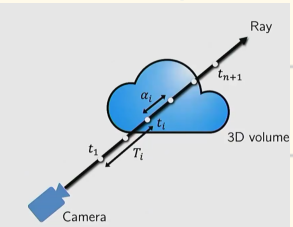
NeRF:

$(x, y, z, \theta, \phi) \rightarrow \square\square\square \rightarrow (RGB\sigma)$

空间坐标 相机视角 F_Θ

Ray Distance：渲染损失，适配渲染模型.

使用 NeRF 渲染：



计算光路上点的 颜色、密度、外观

⇓

计算单条光线在光路上被挡住了多少.

> 从渲染公式中得到 Loss，在二维数据中学习.

Problem： NeRF 本身是隐式表示的，在渲染时需要采样导致速度慢.